Universidad de Antioquia
Departamento de Electronica
Informatica II

Solución propuesta desafio I Informatica II

Jesús Sebastián Mendoza
Danna isabela Ochoa
Facultad de Ingenieria

23 de marzo de 2026

1. Introducción

El presente informe describe el desarrollo del Desafío 1, un proyecto de programación en C++ cuyo objetivo fue implementar un juego basado en Tetris, en el que distintas piezas se desplazan y rotan dentro de un tablero con el propósito de completar y eliminar filas llenas.

Como estrategia principal, se adoptó una estructura modular, dividiendo el sistema en componentes con responsabilidades específicas, tales como la entrada de datos, la gestión de piezas, la lógica del juego, la visualización en consola y el control general del flujo del programa. Esta organización permitió desarrollar una solución más clara, ordenada y fácil de mantener.

Una de las decisiones más importantes del proyecto fue representar tanto el tablero como las piezas mediante operaciones a nivel de bits. Esta elección permitió optimizar el uso de memoria y trabajar con una lógica más eficiente, aunque también exigió un mayor nivel de análisis y precisión en el diseño e implementación de los algoritmos. Por esta razón, durante el desarrollo fue necesario estudiar con detalle qué tipos de datos y qué estructuras de control resultaban más adecuados en cada módulo, buscando siempre un equilibrio entre eficiencia, claridad y corrección.

A lo largo del proceso se enfrentaron distintos retos, entre ellos la validación de las entradas del usuario, la representación compacta de las piezas, la detección de colisiones, la rotación de figuras y la eliminación eficiente de filas completas. Cada uno de estos aspectos requirió decisiones de diseño particulares y una evolución progresiva de la solución hasta alcanzar una versión más organizada, funcional y comprensible.

En este informe se presenta el análisis del problema, las consideraciones tomadas para la alternativa de solución propuesta, el esquema de tareas definido para el desarrollo de los algoritmos, la descripción de los algoritmos implementados, los principales problemas encontrados durante la construcción del proyecto y la evolución de la solución hasta su versión final.

2. Análisis del problema y consideraciones para la alternativa de solución propuesta

El problema planteado en este proyecto consistió en desarrollar, en el lenguaje de programación C++, un juego basado en Tetris que permitiera simular el comportamiento esencial del sistema: crear un tablero de dimensiones variables, generar piezas con distintas formas ya establecidas, permitir su desplazamiento y rotación, detectar colisiones con bordes o con otras piezas, fijarlas en el tablero cuando ya no pudieran seguir bajando y eliminar filas completas. De igual forma, la solución debía cumplir con esta funcionalidad mientras se desarrollaba de la manera más organizada y eficiente posible.

Desde el primer análisis se identificó que el reto, además de ser visual, también era estructural y lógico, por lo cual era necesario definir una representación interna adecuada para el tablero y las piezas. Frente a estas necesidades, se consideraron distintas alternativas de representación. Una opción posible era modelar el tablero mediante una matriz tradicional de enteros o caracteres, donde cada celda indicara si estaba vacía u ocupada. Aunque esta opción resulta intuitiva, se optó por una representación a nivel de bits, representando tanto las filas del tablero como las formas de las piezas mediante bytes. Esta decisión permitió aprovechar mejor la memoria y hacer más eficientes ciertas operaciones, como consultar y comparar celdas mediante máscaras y desplazamientos.

Además, desde la guía del desafío ya se establecía como condición que el ancho del tablero debía ser múltiplo de 8 y que el alto debía ser al menos 8. A partir de esa restricción, se diseñó una solución coherente con dichos requerimientos, aprovechando una representación por bytes en la que cada uno almacena exactamente ocho posiciones. De esta manera, cada fila del tablero pudo organizarse como un conjunto de bytes, facilitando el acceso a una celda específica mediante el cálculo de su byte correspondiente y de la máscara de bit adecuada.

Otra consideración importante fue la organización del proyecto. En lugar de concentrar toda la lógica en un solo archivo, se propuso una arquitectura modular. Esta alternativa fue seleccionada porque permitía separar

claramente las responsabilidades del sistema: un módulo para la entrada de datos, otro para el tablero, otro para las piezas, otro para la lógica del juego, otro para la visualización y, finalmente, uno principal encargado de coordinar todo el flujo. Esta estructura no solo hizo más legible el programa, sino que también facilitó las pruebas, la corrección de errores y el desarrollo ordenado de la solución.

Otra decisión importante dentro de la solución fue el uso de una cola circular para manejar las filas del tablero. En lugar de desplazar físicamente todas las filas cada vez que se elimina una línea completa, se utiliza un índice que marca el inicio lógico del tablero. De esta manera, cuando una fila se limpia, no es necesario mover todos los datos en memoria, sino únicamente actualizar ese índice para reinterpretar cuál es la primera fila visible. Esta estrategia permitió hacer más eficiente la eliminación de filas, reducir operaciones innecesarias y mantener una organización coherente del tablero durante la ejecución del juego.

En cuanto a la lógica de control, se eligieron estructuras como `for`, `while` e `if / else if`, según la naturaleza de cada problema. Los ciclos `for` se utilizaron cuando el número de iteraciones era fijo y conocido, por ejemplo, al recorrer las posiciones de una pieza. Los ciclos `while` se reservaron para validaciones o procesos cuya repetición dependía de una condición variable, como la lectura correcta de datos del usuario o la limpieza de filas consecutivas. Por su parte, las estructuras condicionales permitieron diferenciar acciones, tipos de pieza y validaciones específicas de manera clara.

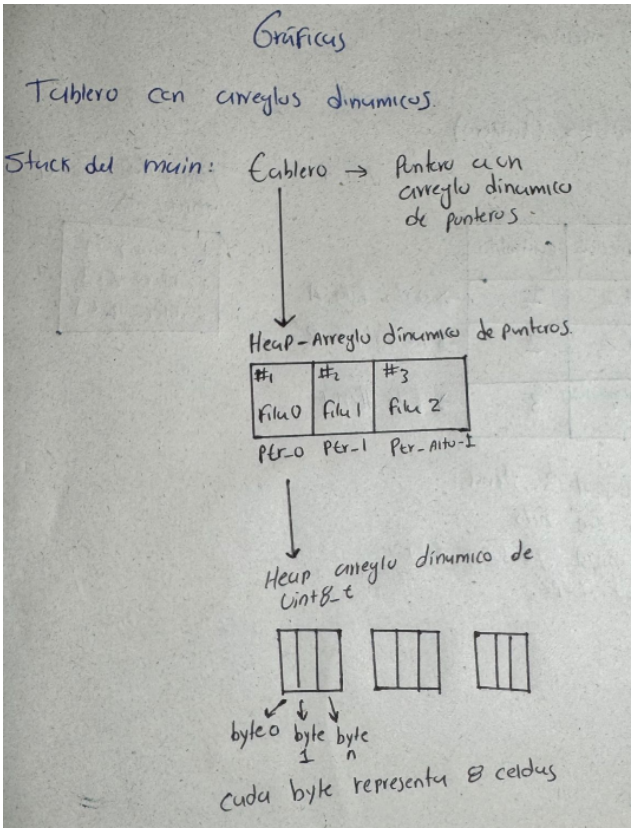

Figura 1: Representación gráfica de la estructura dinámica del tablero.

2.1. Justificación de tipos de datos

Para la selección de los tipos de datos se priorizó el uso eficiente de memoria y la portabilidad entre arquitecturas, utilizando los tipos de ancho fijo definidos en `<cstdint>`.

2.1.1. `uint8_t` — parámetros y variables acotadas sin signo

Se utilizó `uint8_t` para variables cuyo rango está estrictamente limitado entre 0 y 255 y que, además, nunca toman valores negativos. En este proyecto, `ancho` y `alto` están restringidos por validación de entrada a valores máximos de 64 y 40, respectivamente; `bytesPorFila` puede tomar como máximo el valor de `ancho / 8`, es decir, 8; e `inicio` funciona como un índice circular acotado por `alto`. De igual forma, `tipoPieza` toma valores entre 0 y 6, y `rotacionPieza` entre 0 y 3.

El uso de `uint8_t` en lugar de `int` permite reducir el consumo de memoria de 4 bytes a 1 byte por variable, lo cual resulta conveniente en un programa donde varias de estas variables se pasan repetidamente como parámetros entre funciones. También se utilizó este tipo para representar las filas de las piezas (`forma[4]`), ya que cada fila corresponde a un patrón de 8 bits.

2.1.2. `int8_t` — variables con signo acotadas

Se utilizó `int8_t` para `filaPieza` y `colPieza`, ya que estas variables sí pueden tomar valores negativos. Por ejemplo, `colPieza` puede ser negativa cuando una pieza con columnas vacías a la izquierda se ubica cerca del borde izquierdo del tablero. Sus valores se mantienen dentro de rangos pequeños, por lo que caben sin problema en un `int8_t`, cuyo intervalo va de -128 a 127.

Se eligió este tipo en lugar de `int` para mantener coherencia con la estrategia general de ahorro de memoria y porque representa adecuadamente posiciones pequeñas con signo dentro del tablero.

2.1.3. `uint32_t` — semilla del generador aleatorio

La variable `semilla` se declaró como `uint32_t` porque el algoritmo XorShift implementado en `piezaAleatoria` realiza desplazamientos de 13, 7 y 17 bits. Para que dicho algoritmo funcione correctamente, es necesario que opere sobre una variable de exactamente 32 bits.

Usar `unsigned int` no garantiza ese tamaño en todas las arquitecturas, mientras que `uint64_t` implicaría utilizar más memoria de la necesaria. Por esta razón, `uint32_t` fue la opción más adecuada, ya que asegura portabilidad y funcionamiento correcto del generador pseudoaleatorio.

2.1.4. `bool` — funciones con resultado lógico

Se utilizó `bool` en funciones como `hayColision`, `filaLlena`, `celdaOcupada` y `verificarGameOver`, ya que todas ellas retornan un resultado estrictamente lógico: verdadero o falso. Emplear `bool` en lugar de `int` mejora la legibilidad del código y expresa con mayor claridad la intención semántica de cada función.

2.1.5. `int` — variables de control en ciclos

Las variables de control de los ciclos `for` y `while` se mantuvieron como `int`, siguiendo una práctica común en C++. Esto se debe a que el compilador suele promover automáticamente los tipos enteros pequeños a `int` durante operaciones aritméticas y comparaciones, por lo que usar `uint8_t` en índices de iteración no representa una mejora real en memoria y puede generar advertencias o errores sutiles.

Además, usar tipos sin signo en ciclos descendentes puede causar comportamientos incorrectos. Por ejemplo, una estructura como `for(uint8_t i = n; i >= 0; i--)` produce un ciclo infinito cuando `i` llega a 0, debido a que `uint8_t` no puede representar valores negativos.

2.1.6. `char` — entrada del usuario

Se utilizó `char` para la variable `accion` en la función `leerAccion`, ya que esta representa un carácter ingresado por el usuario desde el teclado. `char` es el tipo más adecuado para este propósito, ocupa 1 byte y es totalmente compatible con `cin` y con las comparaciones de caracteres utilizadas para validar las acciones permitidas en el juego.

3. Esquema de tareas definidas en el desarrollo de los algoritmos.

Para desarrollar el proyecto de manera ordenada, se definió un esquema de trabajo por etapas, donde cada tarea respondía a una necesidad concreta del sistema. Esto permitió construir la solución de forma progresiva y organizada.

La primera tarea fue analizar los requerimientos del desafío y las restricciones de la guía. En esta etapa se identificó la necesidad de crear un tablero configurable, manejar piezas con distintas formas, permitir movimientos y rotaciones, detectar colisiones, eliminar filas y mostrar el estado del juego en consola.

La segunda tarea consistió en definir la estructura general de la solución. Para ello se decidió trabajar con una arquitectura modular, separando el sistema en componentes como entrada de datos, tablero, piezas, lógica del juego, visualización y control principal.

La tercera tarea fue diseñar la representación interna del tablero y de las piezas. En esta fase se escogió el uso de arreglos dinámicos y bytes para almacenar la información, además de incorporar la lógica de cola circular para manejar las filas del tablero de manera más eficiente.

La cuarta tarea fue implementar la entrada y validación de datos. Aquí se desarrollaron las funciones necesarias para leer las dimensiones del tablero, verificar que cumplieran las condiciones exigidas y capturar correctamente las acciones del usuario durante el juego.

La quinta tarea consistió en programar la creación y administración del tablero. Se implementaron las funciones para reservar y liberar memoria, consultar si una celda estaba ocupada y marcar posiciones dentro del tablero.

La sexta tarea fue desarrollar los algoritmos relacionados con las piezas. En esta parte se trabajó en la generación de las formas base, las rotaciones, el cálculo de bordes, la selección pseudoaleatoria y la inicialización de nuevas piezas.

La séptima tarea consistió en implementar la lógica principal del juego. En esta etapa se desarrollaron los algoritmos de colisión, fijación de piezas, detección de filas llenas, limpieza de filas y verificación de game over.

La octava tarea fue desarrollar la visualización del tablero en consola. Para ello se creó la función encargada de recorrer el tablero y superponer la pieza activa sobre las piezas fijas, permitiendo mostrar el estado actual del juego de forma clara.

La novena tarea consistió en integrar todos los módulos en el programa principal. En esta fase se coordinó el flujo general del sistema, enlazando la lectura de datos, la creación del tablero, la generación de piezas y la ejecución continua del juego.

Finalmente, otra tarea fundamental fue realizar pruebas constantes y aplicar los ajustes necesarios. A medida que se implementaba cada módulo, se verificó su funcionamiento de forma individual y luego en conjunto con los demás. Estas pruebas permitieron detectar errores, corregir problemas de lógica y hacer mejoras en la organización y comportamiento general del programa.

4. Algoritmos implementados.

4.1. Módulo de Entrada (`input.cpp`)

Este módulo gestiona toda la interacción con el usuario a través de la consola, garantizando que los datos ingresados sean válidos antes de ser procesados por el resto del sistema.

4.1.1. Función `leerDimensiones`

Recibe dos punteros a variables de tipo `uint8_t` y se encarga de capturar las dimensiones del tablero de juego mediante validación estricta por ciclos.

Para el **ancho**, solicita al usuario un valor entero y lo acepta únicamente si cumple tres condiciones simultáneas: ser múltiplo de ocho y estar dentro del rango de 8 a 64. Si el valor no satisface alguna de estas condiciones, el ciclo `while` limpia el buffer de entrada con `cin.clear()` e `cin.ignore()` para descartar entradas inválidas o residuos de lecturas anteriores, y vuelve a solicitar el dato. Una vez validado, el valor se convierte explícitamente a `uint8_t` y se almacena a través del puntero correspondiente.

Para el **alto**, el proceso es análogo pero con una restricción más simple: el valor debe estar entre 8 y 40, sin exigir que sea múltiplo de ningún número.

El uso de `uint8_t` como tipo de destino es una decisión de diseño que limita el consumo de memoria, dado que las dimensiones del tablero nunca superarán el rango de un entero sin signo de 8 bits (0–255).

4.1.2. Función `leerAccion`

Captura un carácter que representa la acción del jugador durante la partida. Antes de leer, limpia el buffer con `cin.clear()` e `cin.ignore()` para evitar que saltos de línea o caracteres residuales de lecturas anteriores interfieran con la lectura.

La función acepta seis acciones válidas, reconociendo tanto mayúsculas como minúsculas, tal como se detalla en la Tabla 1.

Tecla	Acción
`A` / `a`	Mover pieza a la izquierda
`D` / `d`	Mover pieza a la derecha
`S` / `s`	Bajar pieza
`W` / `w`	Rotar a la derecha
`E` / `e`	Rotar a la izquierda
`Q` / `q`	Salir del juego

Tabla 1: Acciones válidas del jugador

Si el carácter ingresado no corresponde a ninguna de estas opciones, el ciclo `while` rechaza la entrada y vuelve a solicitarla hasta obtener una acción válida. Una vez validada, la función retorna el carácter para que sea procesado por la lógica del juego.

4.2. Módulo del Tablero (`tablero.cpp`)

Este módulo gestiona la creación, destrucción y manipulación del tablero de juego. La estructura central es una matriz dinámica de bytes donde cada bit representa una celda del tablero, lo que permite un almacenamiento compacto y eficiente. El tablero se implementa como una cola circular, permitiendo desplazar filas sin reasignar memoria.

4.2.1. Función `crearTablero`

Inicializa toda la estructura del tablero en memoria dinámica. Primero calcula `bytesPorFila` dividiendo el ancho entre 8, dado que cada byte almacena el estado de 8 celdas contiguas. A continuación reserva un arreglo de `alto` punteros, donde cada uno apunta a una fila de `bytesPorFila` bytes. Todas las celdas se inicializan en `0x00`, representando un tablero vacío. El índice `inicio` se establece en 0, marcando el tope lógico de la cola circular.

4.2.2. Función `destruirTablero`

Libera ordenadamente la memoria reservada por `crearTablero`. Primero recorre cada fila y libera su arreglo de bytes con `delete[]`, y luego libera el arreglo de punteros. Finalmente asigna `0` al puntero del tablero para evitar referencias a memoria liberada.

4.2.3. Función `ocuparCelda`

Marca una celda del tablero como ocupada mediante operaciones a nivel de bit. Dado que cada fila es un arreglo de bytes, la columna lógica `col` se descompone en dos índices: `byteIndex = col / 8`, que identifica el byte dentro de la fila, y `bitIndex = col % 8`, que identifica el bit dentro de ese byte. La fila física se obtiene mediante la fórmula de cola circular:

$$\texttt{filaFisica} = (\texttt{inicio} + \texttt{fila}) \quad \text{mód } \texttt{alto}$$

La máscara se construye desplazando el bit más significativo hacia la derecha `bitIndex` posiciones (`0x80` bitIndex), y se aplica con una operación OR bit a bit sobre el byte correspondiente, activando únicamente el bit de interés sin alterar los demás.

4.2.4. Función `celdaOcupada`

Consulta si una celda está ocupada utilizando la misma lógica de indexación que `ocuparCelda`: se calculan `byteIndex`, `bitIndex` y `filaFisica` de forma idéntica, y se construye la misma máscara. En lugar de OR, aplica una operación AND bit a bit sobre el byte correspondiente. Si el bit consultado está activo el resultado es distinto de cero y la función retorna `true`; de lo contrario retorna `false`.

4.3. Módulo de Piezas (`pieza.cpp`)

Este módulo gestiona la representación, generación y transformación de las piezas del juego. Cada pieza se codifica en un arreglo de 4 bytes (`forma[4]`), donde cada byte representa una fila de la matriz 4×4 que contiene la pieza, y cada bit indica si la celda correspondiente está ocupada. Esta representación compacta permite aplicar rotaciones mediante operaciones a nivel de bit sin estructuras de datos adicionales.

4.3.1. Función `generarFormaBase`

Inicializa el arreglo `forma[4]` en cero y luego asigna los patrones de bits correspondientes al tipo de pieza recibido. Los siete tipos siguen la nomenclatura estándar del Tetris y se codifican de la siguiente manera:

4.3.2. Función `rotarDerecha`

Realiza una rotación de 90° en sentido horario sobre la matriz 4×4 representada en `forma[4]`. Guarda los cuatro bytes originales en variables temporales y reconstruye cada byte de la forma rotada extrayendo bits específicos de las filas originales con máscaras y desplazándolos a su nueva posición.

Tipo	Pieza	Patrón
0	I	`forma[0] = 0b11110000`
1	O	`forma[0] = forma[1] = 0b11000000`
2	T	`forma[0] = 0b11100000, forma[1] = 0b01000000`
3	S	`forma[0] = 0b01100000, forma[1] = 0b11000000`
4	Z	`forma[0] = 0b11000000, forma[1] = 0b01100000`
5	J	`forma[0] = 0b11100000, forma[1] = 0b10000000`
6	L	`forma[0] = 0b11100000, forma[1] = 0b00100000`

Tabla 2: Codificación de bits para cada tipo de pieza

La lógica equivale a la transposición clásica de matrices seguida de una inversión de columnas, pero ejecutada completamente con operaciones AND, OR y desplazamiento de bits. Por ejemplo, el bit más significativo del nuevo `forma[0]` proviene del bit más significativo de `f3` (esquina inferior izquierda), que en la rotación horaria pasa a ocupar la esquina superior izquierda.

4.3.3. Función `rotarIzquierda`

Realiza una rotación de 90° en sentido antihorario, equivalente a tres rotaciones a la derecha. Utiliza la misma estrategia que `rotarDerecha` pero con el orden de las filas y los desplazamientos invertidos: el bit más significativo del nuevo `forma[0]` proviene del bit menos significativo de `f0` (esquina inferior derecha en la nueva orientación), reflejando la transposición opuesta.

4.3.4. Función `generarForma`

Combina `generarFormaBase` y las funciones de rotación para obtener la forma de una pieza en una orientación arbitraria. Primero genera la forma base del tipo indicado y luego aplica `rotacion` veces la rotación en el sentido especificado por `sentido` (`'R'` para derecha, cualquier otro valor para izquierda).

4.3.5. Función `maxRotaciones`

Retorna la cantidad de orientaciones distintas de cada tipo de pieza. La pieza O tiene simetría total y solo 1 orientación; las piezas I, S y Z tienen simetría de 180° y 2 orientaciones; las piezas T, J y L no tienen simetría rotacional y requieren 4 orientaciones.

4.3.6. Función `piezaAleatoria`

Genera un índice de pieza pseudoaleatorio entre 0 y 6 mediante el algoritmo *xorshift32*, que opera sobre una semilla de 32 bits aplicando tres operaciones XOR con desplazamientos de 13, 7 y 17 bits respectivamente. Este algoritmo es de periodo largo, liviano y no requiere bibliotecas externas. El resultado se reduce al rango $[0, 6]$ con la operación módulo 7.

4.3.7. Función `iniciarPieza`

Inicializa el estado de una nueva pieza al comenzar su descenso. Genera un tipo aleatorio con `piezaAleatoria`, establece la rotación en 0, posiciona la pieza en la fila 0 y la centra horizontalmente calculando `ancho / 2 - 2`, y finalmente obtiene su forma base con `generarFormaBase`.

4.3.8. Función `offsetBorde`

Calcula la distancia entre el borde de la matriz 4×4 y el bit activo más cercano en el sentido indicado. Para el sentido izquierdo (`'L'`), recorre las columnas de izquierda a derecha y retorna el índice de la primera columna que contenga al menos un bit activo en cualquier fila. Para el sentido derecho, realiza el recorrido inverso y retorna la distancia desde la columna 3. Esta función se utiliza para detectar cuánto puede moverse la pieza antes de alcanzar un borde del tablero.

4.4. Módulo de Juego (`juego.cpp`)

Este módulo implementa la lógica central del juego, incluyendo la detección de colisiones, la fijación de piezas al tablero, la eliminación de filas completas y la verificación de fin de partida. Todas las operaciones trabajan directamente sobre la representación compacta de bits del tablero y hacen uso de la indexación por cola circular definida en el módulo del tablero.

4.4.1. Función `hayColision`

Determina si una pieza en una posición dada colisiona con los bordes del tablero o con celdas ya ocupadas. Primero calcula los offsets de borde izquierdo y derecho con `offsetBorde` para obtener el ancho efectivo de la pieza, descartando el padding de bits vacíos en la matriz 4×4. Con esto verifica que la pieza no exceda los límites horizontales ni que alguna de sus filas sobrepase el fondo del tablero.

La detección de colisiones y la fijación de piezas también fueron refinándose hasta compartir una misma convención de posicionamiento. En la versión final, funciones como `hayColision`, `fijarPieza` e `imprimirTablero` tuvieron que interpretarse bajo la misma lógica de fila, columna, desplazamiento horizontal y ocupación efectiva de la pieza, de manera que una figura se validara, se dibujara y se almacenara exactamente en la misma posición. Esta unificación fue necesaria porque, durante el desarrollo, llegaron a presentarse desajustes entre lo que se mostraba en pantalla y lo que realmente quedaba fijado en memoria. Como consideración de implementación, esta parte del proyecto mostró que, cuando varios módulos operan sobre un mismo estado, la consistencia entre ellos es un requisito estructural y no solo una mejora de legibilidad.

4.4.2. Función `fijarPieza`

Escribe permanentemente los bits de la pieza en el tablero cuando esta no puede descender más. Utiliza la misma descomposición en `parteAlta` y `parteBaja` que `hayColision`, garantizando que las celdas fijadas coincidan exactamente con la posición visualizada. Para manejar el caso en que `col` sea negativo (pieza con padding izquierdo que sobresale del borde), la función opera con aritmética entera con signo y ajusta el desplazamiento correspondientemente. La escritura se realiza con OR bit a bit sobre los bytes del tablero, preservando el estado de las demás celdas.

4.4.3. Función `filaLlena`

Comprueba si una fila lógica del tablero está completamente ocupada. Traduce la fila lógica a su índice físico mediante la fórmula de cola circular y recorre todos sus bytes verificando que cada uno sea igual a `0xFF`, es decir, que todos sus bits estén activos. Si algún byte no cumple esta condición, la función retorna `false` inmediatamente.

4.4.4. Función `limpiarFilas`

Elimina todas las filas completas del tablero sin reasignar memoria, mediante una gestión directa de punteros. Recorre las filas de abajo hacia arriba; al encontrar una fila llena, guarda su puntero en una

variable temporal y limpia sus bytes a `0x00`. Luego desplaza hacia abajo los punteros de todas las filas superiores a la eliminada, de modo que cada posición lógica apunte a la fila que estaba una posición más arriba. Finalmente, coloca el puntero de la fila vaciada en el techo del tablero, reciclándola como la nueva fila superior vacía.

Este enfoque es considerablemente más eficiente que copiar el contenido byte a byte, ya que solo reordena punteros. Además, como el desplazamiento se realiza internamente sin modificar el índice `inicio` de la cola circular, las filas por debajo de la eliminada permanecen intactas. El índice `i` no se decrementa al eliminar una fila, de modo que la posición recién ocupada por la fila superior se reevalúa en la siguiente iteración, permitiendo detectar múltiples filas consecutivas completas.

4.4.5. Función `verificarGameOver`

Evalúa si la partida ha terminado comprobando si la siguiente pieza colisiona desde su posición inicial. Genera la forma base del tipo de pieza entrante, la ubica en la fila 0 centrada horizontalmente (`ancho / 2 - 2`) y delega la comprobación a `hayColision`. Si hay colisión en esta posición, significa que el tablero ha llegado al tope y la función retorna `true`.

4.5. Módulo de Visualización (`display.cpp`)

Este módulo se encarga de renderizar el estado del juego en la consola, dibujando el tablero con sus celdas ocupadas y la pieza activa en su posición actual, así como el mensaje de fin de partida.

4.5.1. Función `imprimirTablero`

Dibuja el tablero completo en la consola recorriendo cada celda de la cuadrícula lógica. Primero imprime el borde superior como una línea de guiones delimitada por los caracteres `+`. Luego, para cada fila, imprime el borde izquierdo y recorre todas las columnas evaluando dos condiciones independientes por celda.

La primera condición consulta si la celda está ocupada por una pieza ya fijada, delegando esta verificación a `celdaOcupada` del módulo del tablero. La segunda condición determina si la celda pertenece a la pieza activa en caída: calcula las coordenadas relativas de la celda respecto a la posición de la pieza (`filaRel = i - fila`, `colRel = j - col`) y, si ambas están dentro del rango $[0, 3]$, construye una máscara (`0x80` colRel) y la aplica con AND sobre el byte `forma[filaRel]` para verificar si el bit correspondiente está activo.

Si alguna de las dos condiciones es verdadera, se imprime el carácter `#` seguido de un espacio; de lo contrario se imprime un punto. Al finalizar cada fila se añade el borde derecho, y al terminar todas las filas se imprime el borde inferior con el mismo formato que el superior.

4.5.2. Función `imprimirGameOver`

Extiende `imprimirTablero` para mostrar el estado final de la partida. Invoca `imprimirTablero` con los mismos parámetros para mostrar el tablero en su estado al momento del fin de juego y, a continuación, imprime el mensaje `===== GAME OVER =====` en la consola.

4.6. Módulo Principal (`main.cpp`)

Este módulo integra todos los módulos del sistema y contiene el ciclo principal de juego. Su responsabilidad es coordinar la inicialización, el flujo de interacción con el usuario y la secuencia de eventos que ocurren en cada turno, delegando cada tarea al módulo especializado correspondiente.

4.6.1. Inicialización

Se declaran todas las variables de estado del tablero y de la pieza activa con valores iniciales en cero. La semilla del generador pseudoaleatorio se inicializa con el tiempo del sistema mediante `time(0)`, garantizando una secuencia de piezas distinta en cada partida. A continuación se solicitan las dimensiones del tablero con `leerDimensiones`, se reserva la memoria con `crearTablero` y se genera la primera pieza con `iniciarPieza`.

4.6.2. Ciclo Principal de Juego

El juego se ejecuta dentro de un ciclo `while` controlado por la bandera `juegoActivo`. En cada iteración se lleva a cabo la siguiente secuencia de eventos:

En primer lugar, se renderiza el estado actual del tablero con la pieza en su posición corriente mediante `imprimirTablero`. Luego se captura la acción del jugador con `leerAccion` y se evalúa con una cadena de condicionales:

- **Movimiento lateral** (`A`/`D`): se invoca `hayColision` con la columna decrementada o incrementada en uno; si no hay colisión, se actualiza `colPieza`.
- **Descenso manual** (`S`): se verifica colisión con la fila siguiente y, de no haberla, se incrementa `filaPieza`.
- **Rotación** (`W`/`E`): la rotación se aplica sobre una copia temporal `formaTest` con `rotarDerecha` o `rotarIzquierda`. Solo si la forma rotada no produce colisión se transfiere a `formaActual`, evitando rotaciones inválidas sin modificar el estado hasta confirmar la viabilidad.
- **Salida** (`Q`): establece `juegoActivo` en `false` para terminar el ciclo.

4.6.3. Fijación de Piezas

Tras procesar la acción del jugador, el módulo aplica la gravedad del juego. Si la pieza no puede descender una fila más, se ejecuta la siguiente secuencia: se fija la pieza en el tablero con `fijarPieza`, se eliminan las filas completas con `limpiarFilas`, se genera una nueva pieza con `iniciarPieza` y se verifica inmediatamente si la nueva pieza colisiona en su posición inicial con `verificarGameOver`. Si hay colisión, se muestra el estado final con `imprimirGameOver` y se termina el ciclo.

4.6.4. Liberación de Memoria

Al salir del ciclo, independientemente de si la partida terminó por game over o por acción del jugador, se invoca `destruirTablero` para liberar toda la memoria dinámica reservada durante la partida.

5. Problemas de Desarrollo

A lo largo del proyecto surgieron diversas dificultades técnicas que requirieron decisiones de diseño cuidadosas. El primer reto fue la representación compacta del tablero en bits: almacenar cada fila como un arreglo de `uint8_t` mejoró el uso de memoria, pero impuso restricciones como exigir que el ancho fuera múltiplo de 8 y obligó a calcular con precisión el byte y el bit correspondientes a cada celda en toda operación de lectura, escritura o verificación.

La gestión dinámica de memoria también presentó complejidades, tanto en la construcción del arreglo de filas como en su correcta liberación y reorganización al eliminar líneas completas. Esto se vio agravado por la implementación del tablero como cola circular, que introdujo una indirección constante entre la fila lógica y su posición física en memoria, obligando a que todas las funciones aplicaran consistentemente la fórmula `(inicio + fila) % alto`.

La eliminación de filas completas resultó especialmente delicada: no bastaba con borrar su contenido, sino que era necesario lograr que las filas superiores descendieran correctamente sin alterar las inferiores. Durante el desarrollo se presentaron fallas en las que algunas filas no bajaban como correspondía, lo que evidenció inconsistencias en la lógica de desplazamiento interno. La solución basada en reordenamiento de punteros fue más eficiente que copiar bytes, pero exigió un manejo muy preciso para no modificar referencias que debían permanecer intactas.

La detección de colisiones y la rotación de piezas fueron también fuentes frecuentes de errores. Ambas operan directamente sobre bits, lo que las hace eficientes pero propensas a inconsistencias: las colisiones requerían dividir la comparación entre bytes consecutivos del tablero, mientras que las rotaciones debían reconstruir cada fila manualmente a partir de columnas de la forma original. En ambos casos, un error menor en las máscaras o los desplazamientos producía comportamientos incorrectos difíciles de depurar.

A esto se sumó la dificultad de manejar correctamente los bordes del tablero y las posiciones especiales de algunas piezas. En ciertos casos, una figura podía tener columnas vacías a la izquierda o a la derecha, lo que obligó a calcular su borde efectivo real en lugar de trabajar únicamente con el bloque completo de 4 por 4. También fue necesario contemplar posiciones horizontales negativas para mantener un comportamiento coherente cerca del borde izquierdo.

Finalmente, mantener la coherencia entre módulos fue un desafío transversal a todo el desarrollo. La visualización, la detección de colisiones y la fijación de piezas debían compartir exactamente la misma interpretación del estado del tablero; cualquier discrepancia entre ellas se manifestaba como errores visibles en el comportamiento del juego. Gran parte del trabajo de desarrollo consistió en unificar esta lógica entre los distintos módulos para asegurar que lo que se mostraba en pantalla coincidiera con lo que realmente se almacenaba y procesaba en memoria.

6. Evolución de la Solución y Consideraciones para la Implementación

La solución del proyecto evolucionó desde una idea general de reproducir la lógica básica del juego hasta una implementación modular orientada a la eficiencia en memoria y al control detallado del tablero. En las primeras etapas, uno de los aspectos más importantes fue definir una representación interna adecuada, ya que muchas decisiones posteriores dependían directamente de cómo se almacenaría el estado del juego. A partir de esa necesidad, se adoptó una estructura basada en bits, en la cual cada fila del tablero se representa mediante arreglos de `uint8_t`. Esta decisión permitió un uso compacto de la memoria, aunque también incrementó la complejidad de las operaciones básicas sobre el tablero.

A medida que el desarrollo avanzó, se hizo evidente que no bastaba con lograr que cada módulo funcionara de forma aislada; era necesario que todos compartieran exactamente la misma interpretación del tablero y de la pieza activa. Por ello, la solución fue evolucionando hacia una organización más clara por módulos, separando responsabilidades entre entrada, manejo del tablero, gestión de piezas, lógica del juego, visualización y control general desde `main`. Esta separación facilitó la depuración y permitió corregir errores de manera más localizada.

En el caso del tablero, la solución evolucionó desde la necesidad inicial de almacenar filas de manera compacta hasta una implementación más flexible basada en memoria dinámica y en un arreglo de punteros. Durante el desarrollo se hizo evidente que no bastaba con guardar el contenido de cada fila, sino que también era necesario poder reorganizarlas eficientemente al eliminar líneas completas. Por esta razón, el tablero terminó definiéndose como una estructura dinámica capaz de integrarse con la lógica de cola circular mediante la variable `inicio`. Esta decisión permitió separar la posición visual de las filas de su ubicación física en memoria, aunque también obligó a que todas las funciones del sistema accedieran al tablero bajo una misma convención. Como consideración de implementación, esta parte del desarrollo mostró que, cuando se trabaja

con estructuras circulares y memoria dinámica, es indispensable definir desde el comienzo una regla única de acceso a los datos y conservarla de forma consistente en todos los módulos.

La limpieza de filas también evolucionó hacia una estrategia más coherente con la representación dinámica del tablero. En lugar de desplazar el contenido de cada fila byte por byte, la solución final reutilizó la fila eliminada y reordenó los punteros del arreglo principal, conservando intactos los datos que no debían modificarse. Esta decisión redujo el costo interno de la operación y, al mismo tiempo, corrigió problemas observados durante el desarrollo, en los que algunas filas no descendían de forma consistente después de eliminar una línea. Como consideración de implementación, este cambio mostró que optimizar una estructura de datos no consiste únicamente en reducir operaciones, sino en garantizar que la reorganización interna preserve exactamente el comportamiento visual esperado.

En cuanto a las piezas, la evolución de la solución estuvo marcada por la necesidad de representar, rotar y posicionar figuras de manera coherente con la estructura binaria del tablero. Inicialmente, la principal dificultad fue lograr rotaciones correctas sin perder consistencia entre orientación, colisión y visualización. La solución final se apoyó en operaciones bit a bit y en el uso de bordes efectivos para determinar con mayor precisión el espacio real ocupado por cada pieza. Esto dejó como consideración que, cuando se trabaja con representaciones compactas, no siempre basta con considerar el tamaño teórico de una estructura; en muchos casos es necesario calcular su ocupación efectiva para resolver correctamente movimientos y colisiones.

La detección de colisiones y la fijación de piezas también fueron refinándose durante el desarrollo. Se hizo necesario unificar los criterios utilizados por las funciones que validaban movimientos, dibujaban la pieza activa y la almacenaban finalmente en el tablero. De esta evolución se desprende una consideración clave: en sistemas con varios módulos que manipulan el mismo estado, la coherencia entre visualización y almacenamiento interno es tan importante como la corrección individual de cada función. Un sistema puede parecer correcto en una parte del proceso, pero fallar si otro módulo interpreta las posiciones de forma distinta.

Otra consideración relevante durante la implementación fue el manejo de casos límite. La validación de dimensiones, las posiciones cercanas a los bordes, las columnas vacías en algunas piezas, las posiciones horizontales negativas y la aparición de nuevas piezas en un tablero parcialmente lleno mostraron que gran parte de la estabilidad del programa depende de contemplar explícitamente estas situaciones. En consecuencia, la evolución de la solución no consistió solo en agregar funcionalidades, sino también en robustecer el comportamiento del sistema frente a condiciones especiales.

En términos generales, la implementación permitió concluir que una solución eficiente no surge únicamente de elegir buenas estructuras de datos, sino de mantener coherencia entre ellas, validar cuidadosamente los casos extremos y probar de forma constante cada cambio realizado. La evolución del proyecto demostró que muchas decisiones que en principio parecían puramente técnicas, como usar bits, punteros o una cola circular, terminaron influyendo directamente en la claridad del diseño, en la dificultad de depuración y en la confiabilidad del comportamiento final del juego.